UNO

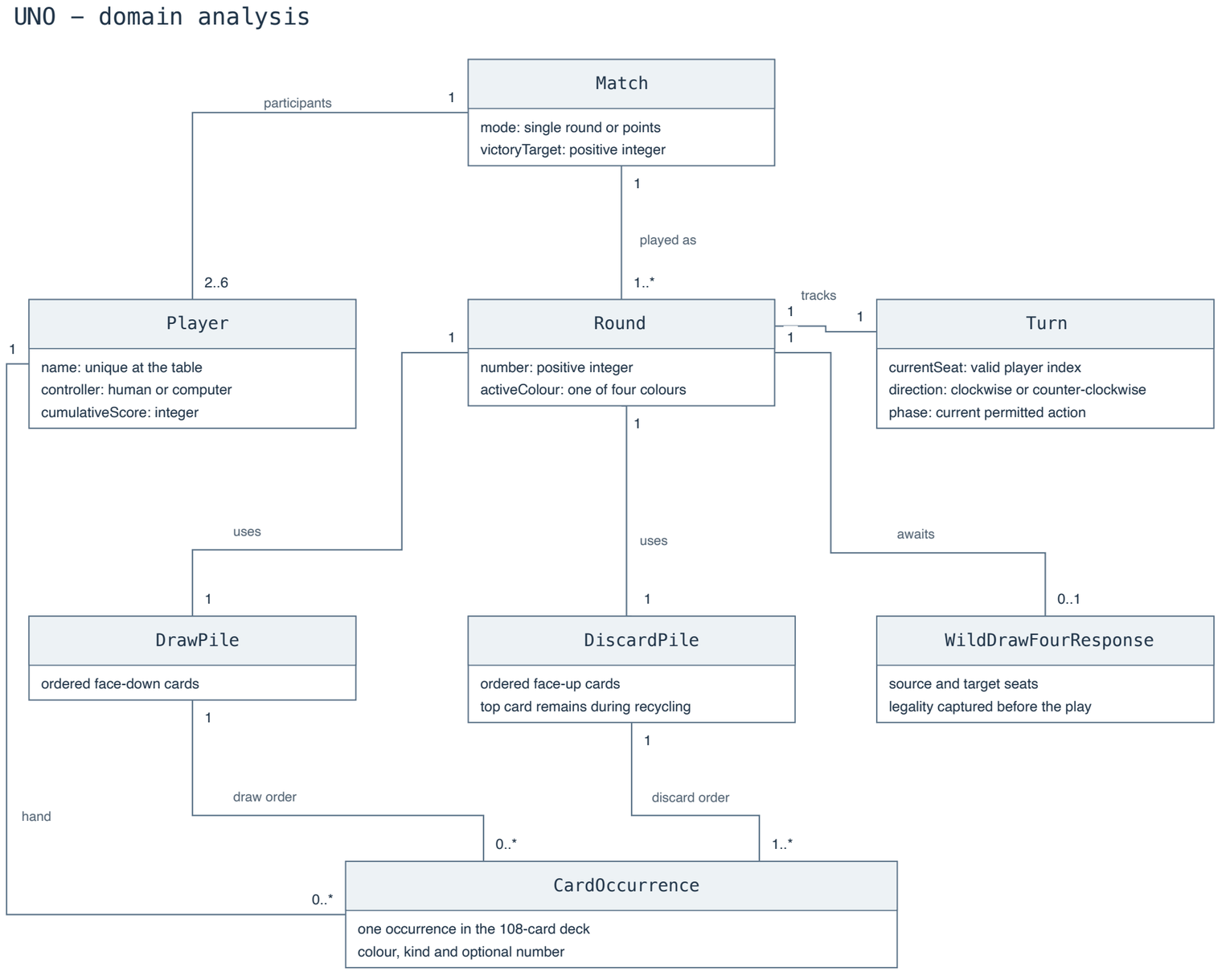
Class specification and use cases

This guide describes the current Java 17 Swing implementation. It presents the domain constraints, contracts of the main operations, computer-player policies, and the navigation between interface views.

Contents

Domain UML 2
Implementation UML 3
Scope and notation 4
Card class specification 5
GameConfig and Player specification 6
Deck class specification 7
GameEngine class specification 8
BotStrategy and BotController specification 11
Interface use cases 12
Interface navigation wireframe 13

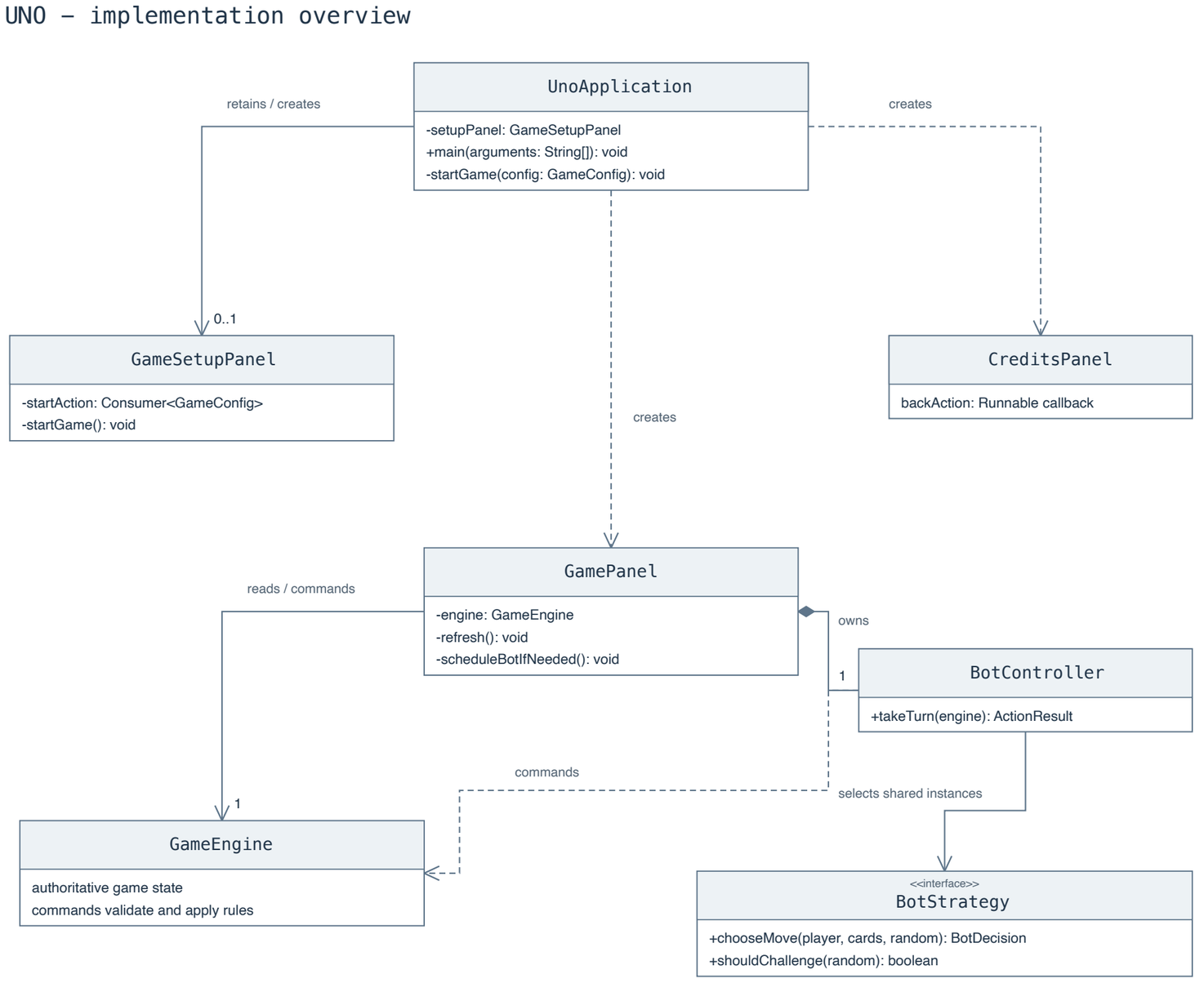
Domain UML



Solid arrow: reference Dashed arrow: dependency Filled diamond: ownership Hollow triangle: interface

Analysis concepts, not Java class declarations. Match, Round and Turn map to GameEngine state.
CardOccurrence describes a deck occurrence; Java Card is a value without a unique physical ID.
Invariant: cards in hands + draw pile + discard pile = 108.

Implementation UML



Solid arrow: reference Dashed arrow: dependency Filled diamond: ownership Hollow triangle: interface

Core has no dependency on Swing, UI or bot implementations.
The setup and credits callbacks navigate through UnoApplication.
See core, bot and UI views for the complete set of 30 top-level source types.

Scope and notation

This guide specifies the current Java 17 Swing implementation: two to six human or computer players, a single round or a points match, UNO declarations, and Wild Draw Four challenges. Match state remains in memory. Saving, loading, configurable house rules, network play, and simulation statistics are outside this implementation.

The domain model distinguishes a match, a round, players, hands, and piles. Match and round behaviour are implemented together by `GameEngine`; there are no separate Java classes named `Match` or `Round`.

`before(x)` and `after(x)` denote the values before and after an accepted operation. `hand(p)` is the ordered hand of player `p`; `size` counts card occurrences, including duplicate values. `current` denotes the current player and `next(p)` the next seat in the active direction. Invariants apply to valid states produced by the normal engine path, at operation boundaries.

Commands return `ActionResult(accepted, message)`. Checked preconditions failing during a normal command produce rejection before the move is applied; this is not an automatic transaction or a rollback guarantee for arbitrary exceptions. Constructors reject invalid arguments with standard Java exceptions. Queries do not change the game state.

Card class specification

[V.Card.valid_value]

For every card `c`, `color(c)` and `kind(c)` are non-null. The following equivalences and implications hold:

- `kind(c) = NUMBER` implies `number(c)` exists and `0 <= number(c) <= 9`.
- `kind(c) != NUMBER` implies `number(c) = null`.
- `kind(c)` is Wild or Wild Draw Four if and only if `color(c) = NONE`.

`NONE` describes a wild's intrinsic absence of colour. It is not a selectable active colour. The colour chosen after a wild belongs to the engine state.

Cards are immutable record values. Two red fives compare equal even when they are two deck occurrences; no physical identifier distinguishes them. A hand is therefore a list, not a set. Equality also makes identical copies interchangeable during the drawn-card choice.

Card.number(color, value): Card

Preconditions

`color` is a real colour and `value` is between 0 and 9.

Postconditions

returns a valid number card with those values.

Card.action(color, kind): Card

Preconditions

`color` is a real colour; `kind` is Skip, Reverse, or Draw Two.

Postconditions

returns a valid coloured action with no numeric component.

Card.wild(kind): Card

Preconditions

`kind` is Wild or Wild Draw Four.

Postconditions

returns a valid wild with colour `NONE` and no number.

pointValue(): int

Postconditions

returns the numeric value for a number card, 20 for a coloured action, and 50 for either wild. `label()` returns the corresponding English display text.

GameConfig and Player specification

[V.GameConfig.valid_seats]

The mode is non-null; the ordered list contains between two and six player configurations; the target is positive, even when the mode is a single round. The default target is 500. Names are non-null and non-blank. No two names compare equal after `trim()` and lower-casing with `Locale.ROOT`.

A bot requires a non-null `BotProfile`. A human's stored bot profile is normalised to null. `GameConfig` copies the supplied list, so changing the caller's list cannot change the configuration. `Player` trims the displayed name when it is created. These constructor checks do not define an exhaustive policy for arbitrary control characters.

Player(config): Player

Preconditions

a non-null valid player configuration.

Postconditions

creates a player with its name and controller choice, an empty hand, and score zero. Hands and scores subsequently belong to the engine's match state.

hand(): List

Postconditions

returns an unmodifiable live view of the hand. It prevents changes through that view but reflects later engine changes; it is not a frozen snapshot. `handSize()` returns the number of occurrences. The engine's package-private mutators add/remove cards, clear a hand, and accumulate scores; GUI and bot code cannot call those mutators directly.

Deck class specification

[V.Deck.standard_composition]

A standard deck contains exactly 108 occurrences: 76 number cards, 24 coloured actions, and eight wilds. Each real colour has one zero, two copies of each number 1–9, and two of each coloured action. There are four Wilds and four Wild Draw Fours.

The draw pile is an `ArrayDeque<Card>`; the discard pile is an `ArrayList<Card>`. In a running round:

`sum(size(hand(p)) for every player p) + size(drawPile) + size(discardPile) = 108`.

`topCard` refers to an occurrence already in the discard pile; `drawnCard` refers to one already in a hand. Neither is counted again.

Deck.standard(random): Deck

Preconditions

a non-null random source.

Postconditions

returns a shuffled classic deck with 108 drawable cards and no discards. Supplying the same random sequence makes shuffling reproducible; ordinary games use a fresh generator.

draw(): Card or null

Postconditions

removes and returns the first drawable card. If the draw pile was empty and at least two discards existed, reshuffles every discard except the visible top into the draw pile first. If no card is available, returns null. The total occurrence count is preserved.

discard(card): void

Preconditions

a non-null card supplied by an operation that transfers it from its previous location.

Postconditions

appends the card to the discard pile. `Deck` itself does not remove it from a player: `GameEngine` coordinates that transfer.

revealStartingNumberCard(): Card

Preconditions

initial dealing has left a drawable number card, as guaranteed by the standard deck and two to six seven-card hands.

Postconditions

returns and discards the first number card encountered. Earlier action/wild cards are returned to the bottom of the draw pile. This is the documented opening simplification; opening-card action effects are not applied.

GameEngine class specification

[V.GameEngine.coherent_turn]

`0 <= currentPlayerIndex < size(players)`. Direction has step +1 or −1, and index wraparound uses `Math.floorMod`. The active colour is one of red, yellow, green, or blue. Exactly one `GamePhase` is active:

Phase	Required main interaction
TURN	Play a matching card or draw once.
DRAWN_CARD_AVAILABLE	Play the drawn value or keep it and pass.
AWAITING_WILD_DRAW_FOUR_RESPONSE	Accept or challenge the pending +4.
ROUND_OVER	Start the next round of an unfinished points match.
GAME_OVER	End this match; New game creates a different engine.

The drawn-card phase retains the relevant card reference. The +4 phase retains source/target indices and captured legality, and its recipient is current. A match winner is available at `GAME_OVER`. The last round result is set after completion and cleared by the next deal.

[V.GameEngine.uno_opportunity]

At most one missed-UNO opportunity is retained. Its target has reached one card without declaring; only another participant can catch it, and never after round or game completion. The opportunity is separate from the main phase and has no wall-clock deadline.

GameEngine(config, random): GameEngine

Preconditions

valid non-null configuration and random source. The one-argument constructor creates a fresh random source.

Postconditions

creates the players, starts round 1, deals seven cards to each, reveals a coloured number, sets its colour active, sets clockwise direction and seat `0` as current, and enters `TURN`. Scores initially equal zero. Seat `0` also starts subsequent rounds.

isPlayable(card): boolean

Preconditions

a non-null valid card.

Postconditions

true exactly when the card is a wild, matches the active colour, matches the visible number, or matches the visible action symbol. This checks compatibility only; it does not check phase, possession, or the Wild Draw Four legality condition.

playableCards(): List

Postconditions

filters the current hand in `TURN`, considers only the drawn value in `DRAWN_CARD_AVAILABLE`, and returns an empty list in other phases. A +4 may appear even when it would be illegal: it is an attempt which an opponent can challenge, not a declaration that the play is legally justified.

playCard(card, selectedColor, declaredUno): ActionResult

Preconditions

phase is TURN or DRAWN_CARD_AVAILABLE; the hand contains the value; the card is compatible; in the drawn-card phase it equals drawnCard; a wild has a selected real colour, neither null nor NONE.

Postconditions

expires the previous UNO opportunity; captures +4 legality from the old hand and old active colour; transfers one matching occurrence from the hand to the discard pile; installs the top card and active colour; then handles UNO, card effects, and completion in that order. A declaration is meaningful only when one card remains. Missing it opens an opportunity for opponents.

Card	Effect before possible completion
Number	Advance one seat.
Skip	Skip the next seat, advancing two.
Reverse	Reverse direction and advance one; with two players advance two, so the actor plays again.
Draw Two	The next player draws up to two available cards and is skipped.
Wild	Keep the chosen colour and advance one.
Wild Draw Four	Keep the chosen colour, retain the evidence, make the recipient current, and wait for a response.

There is no stacking. An empty hand ends the round only after its outstanding effects are resolved. A final +2 therefore applies its penalty before scoring; a final +4 waits for acceptance or challenge. A non-wild ignores the selected-colour argument.

drawCard(): ActionResult

Preconditions

phase is TURN. The player may draw even when already holding a compatible card.

Postconditions

closes the old UNO opportunity and attempts one draw. A compatible draw is added to the hand and opens DRAWN_CARD_AVAILABLE without advancing. An incompatible draw is added and advances the turn. If no card exists, the turn passes with no addition. Only the drawn value can be played after this command.

passAfterDrawing(): ActionResult

Preconditions

phase is DRAWN_CARD_AVAILABLE.

Postconditions

leaves the hand unchanged, clears the drawn restriction, and advances one seat in TURN. This is not a general pass before drawing.

canCatchUno(callerIndex): boolean

Postconditions

true exactly when an opportunity exists, the caller is a valid opposing seat, and the match is not in ROUND_OVER or GAME_OVER. The caller need not own the current main turn. canCurrentPlayerCatchUno() applies this query to the current seat.

catchUno(callerIndex): ActionResult

Preconditions

canCatchUno(callerIndex) is true. The no-argument overload uses the current seat.

Postconditions

the target draws up to two available cards; the opportunity closes; the main turn does not advance. This supports a skipped opponent catching UNO in a two-player game. Accepted play, draw, +4 acceptance, or challenge expires an earlier opportunity; an unavailable command rejected before that step does not.

acceptWildDrawFour(): ActionResult

Preconditions

phase is AWAITING_WILD_DRAW_FOUR_RESPONSE.

Postconditions

closes the UNO opportunity; the recipient draws up to four cards; clears the pending response. If the source's hand is empty, scores and completes the round; otherwise skips the recipient and resumes TURN at the next seat. The selected colour remains active.

challengeWildDrawFour(): ActionResult

Preconditions

the +4 response phase is active and the pending record exists.

The captured play was legal exactly when the source had no card of the colour active immediately before the +4. A matching number or symbol in another colour does not make it illegal.

Postconditions

if the play was illegal, the source draws up to four cards and the challenger retains the turn in TURN. If it was legal, the challenger draws up to six cards and is skipped, or the round completes if the source is empty. Clears the response and prior UNO opportunity. The +4 stays discarded and its selected colour stays active: a successful challenge does not undo the whole preceding move.

A +4 played from a one-card hand is necessarily legal under this condition: the hand cannot also contain a coloured card.

completeRound(winner): internal operation

Trigger

an empty hand after immediate effects and any pending +4 response are resolved.

Postconditions

awards only the winner the sum of all opponents' remaining card values. Stores RoundResult and clears pending +4 and UNO state. A single round enters GAME_OVER. A points match enters ROUND_OVER below its target, or GAME_OVER when the updated winning score reaches the target. The highest total determines the match winner; equal totals keep the first setup seat. Normally only the round winner can newly cross the target because only that player's score changes.

startNextRound(): ActionResult

Preconditions

phase is ROUND_OVER.

Postconditions

increments the round number, creates and shuffles a fresh standard deck, clears and redeals hands, and resets direction, current seat, colour, phase, UNO, drawn choice, pending response, and last result. Keeps names, controller choices, configuration, and accumulated scores. A finished match cannot be continued with this operation.

events(): List

Postconditions

returns an unmodifiable live view of the latest at most 80 event messages. This is neither a full permanent history nor sufficient data for a replay. The engine assumes sequential calls; the public API does not promise thread safety or hidden information against arbitrary Java clients.

BotStrategy and BotController specification

chooseMove(player, playable, random): BotDecision

Preconditions

non-null own player and random source, and a non-empty list of candidates belonging to that player's permitted choice.

Post for each supplied strategy: returns one listed card, a real colour for a wild or null for a non-wild, and `declaresUno = (handSize before play = 2)`. Does not modify the game. The immutable `BotDecision` carries this choice; the engine still validates the resulting command.

Shared correctness policy

`BotController.takeTurn(engine)` first catches a missed UNO on behalf of an eligible bot, even if a human has the main turn. For an active bot it then answers a pending +4 or prepares candidates. It excludes +4 attempts while that bot owns the active colour. With no candidate it draws in `TURN` or passes in the drawn-card phase; otherwise it invokes the selected strategy and submits `playCard`.

All supplied profiles avoid illegal +4 attempts, always declare UNO when required, and use their own hand rather than opponents' cards. Strategy instances are shared and stateless. Challenge probabilities are fixed policies, not deductions from hidden legality.

Profile	Card selection and challenge probability
Random	Uniform over candidate entries and real colours; challenges with probability 15%. Duplicate candidate values retain their separate entries.
Greedy	Wild Draw Four > Draw Two > Skip > Reverse > numbers 9..0 > ordinary Wild; challenges with probability 35%.
Conservative	Above three own cards: numbers 9..0 > coloured actions, tied > Wild > +4. At three or fewer: +4 > Wild > Draw Two > Skip > Reverse > numbers 9..0. Challenges with probability 10%.
Aggressive	+4 > Draw Two > Skip > numbers 9..0 > Reverse > ordinary Wild; challenges with probability 50%.

Greedy, Conservative, and Aggressive choose the most common real colour in their own hand. Ties use red, yellow, green, blue; an all-wild hand defaults to red. Card-ranking ties retain the first candidate in hand order. These are local heuristics, not optimal search or learned policies.

`hasPendingAction(engine)` is true if an eligible bot can catch UNO or the current bot has a turn, drawn-card choice, or +4 response to make. The Swing panel schedules each step with a non-repeating 1.5-second timer. A playable draw and the following choice can occupy separate delayed steps.

Interface use cases

Configure and start a game

Actor: a local human. Pre: the home/setup screen is displayed. Select mode, number of seats, names, human/bot controllers, profiles, and target. Post: typed spinner values are committed and a valid `GameConfig` starts a new engine and game panel. Invalid configuration is reported in the form without starting a game.

Take a human turn

Actor: the current human. Pre: `TURN` or the drawn-card phase. Select Declare UNO before a play leaving one card; click a highlighted value or draw; if drawing yields a candidate, play it or keep it and pass. Post: the engine accepts or rejects the command, and the panel refreshes from its state.

Choose a wild colour

Actor: the current human. Pre: a wild card was selected. The modal colour dialog offers only four real colours. Confirming submits the play; cancelling does not submit that play. This is a transient dialog, not a separate `GamePhase`.

Accept or challenge a +4

Actor: the current recipient, human or bot. Pre: a pending response. Post: the appropriate penalty and continuation/completion are resolved by the captured legality. Human buttons appear in the game's context area; this is a state of the game panel, not a new top-level window.

Continue a points match or return home

Actor: a local human. At `ROUND_OVER`, Next round calls `startNextRound`. At `GAME_OVER`, the final result remains visible. New game returns to setup; choosing Start creates a fresh engine, rather than resetting the existing one. Removing the game panel stops its bot timer and stale callbacks cannot continue its match.

Open Credits and return

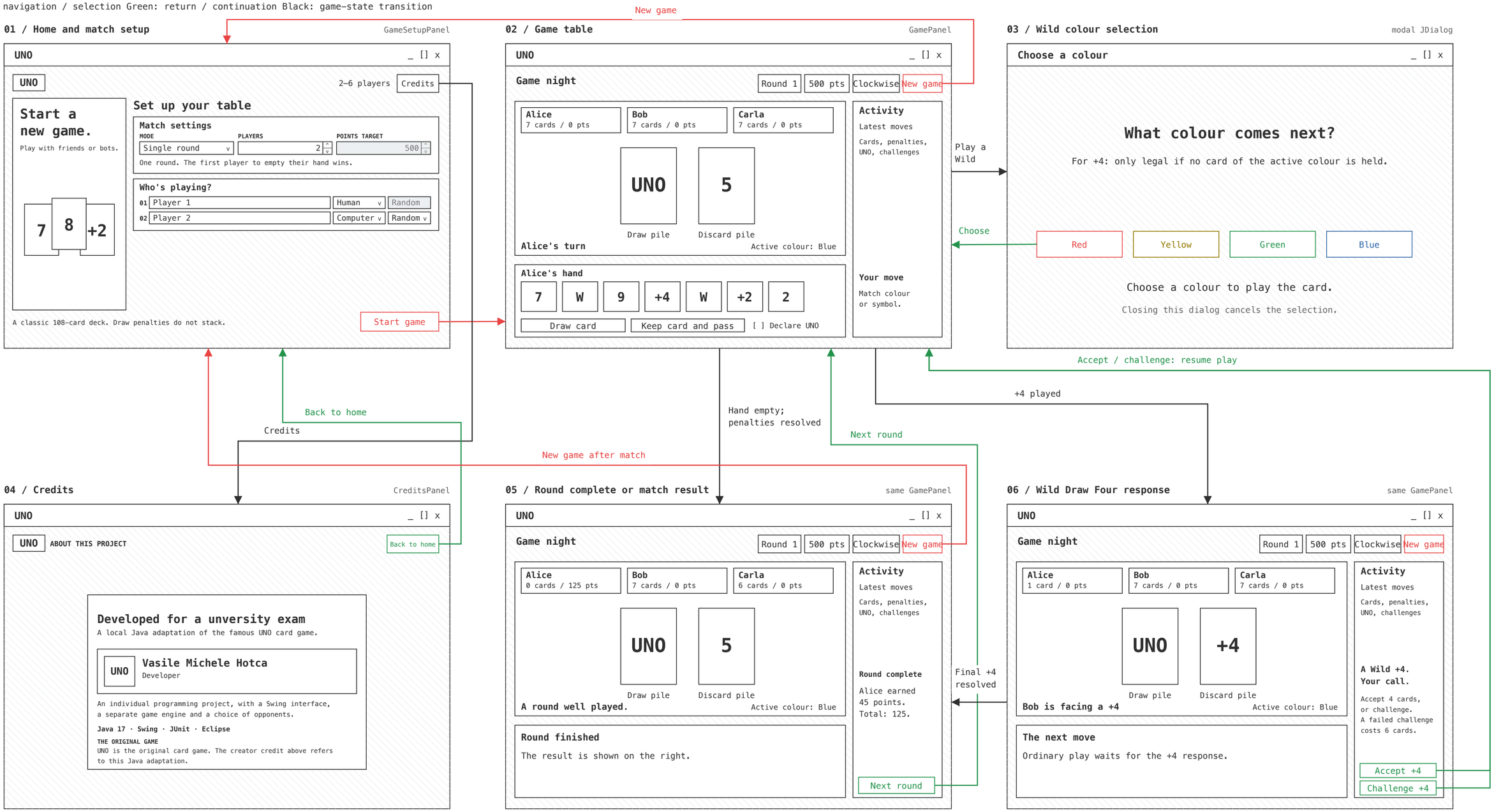
Actor: a local human on home. Credits replaces the current content with `CreditsPanel`; Back to home restores the same configured setup panel. Names and selections are preserved. This screen navigation does not start a game or alter game rules.

The application uses one `JFrame`, replaces its content panel for home, Credits, and play, and uses a modal dialog for wild colour choice. Game results and +4 responses are alternative contents of the game sidebar. A navigation wireframe can show them separately to explain the flow, provided they are labelled as views of that same window.

Interface navigation wireframe

UNO – interface navigation wireframe

Red: navigation / selection Green: return / continuation Black: game-state transition



The response and result views are states of the same game window, not additional application windows.

At GAME_OVER, the result shows the winner and Next round is absent. Closing the colour dialog cancels without playing.

Crossing lines of different colours are separate transitions. All controls and rules follow the existing Swing program.